

MOUNT KENYA UNIVERSITY

ADVANCED WEB DESIGN AND DEVELOPMENT

BIT3208

EDWIN SALATON MEITEIKINI

BSCCS/2024/44160

MAY 2024

YEAR 4 SEM II

Supervised by Lecturer

NYORO MICHAEL

Project title

ECOMMERCE WEBSITE FOR CLICK &GO AUTO SUPPLIERS

This project is build using Html, tailwind CSS and JavaScript (Frontend) PHP (Backend) and MySQL (Database)

Project link

<https://github.com/luke8089/Click-and-Go.git>

or

<https://www.meiteikiniedwin.tech/>

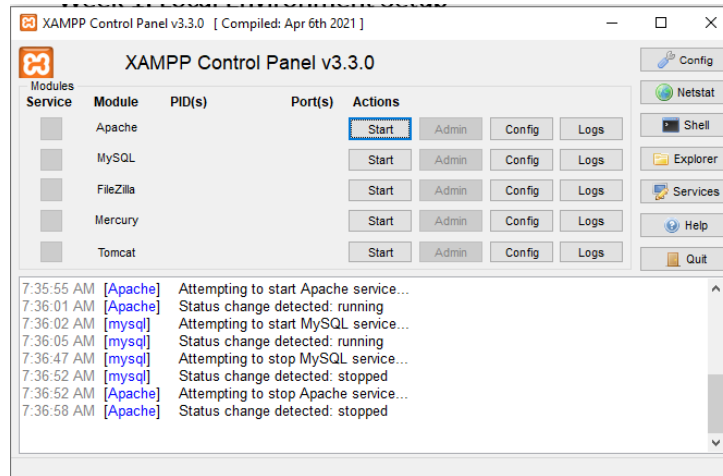
Contents

WEEK 1	3
Local Environment Setup	3
Fig 1: Installation of XAMP	3
Fig 2: Apache and MySQL Running	3
Fig 3: Localhost Test Page	4
Fig 4: Hello World Test	4
Fig 5: Database Connection Test	5
Week 2	6
User Interface Planning and System Design	6
Used Framer for this project	6
Wireframes and GUI Design	6
Fig 1: Hand-drawn Wireframe	6
Fig 2: Dashboard Layout Design	7
Fig 3: Mobile View Mockup	8
Fig 4: Color Theme and Navigation Structure	9
Fig 5: GUI Prototype in Figma/Canva/Draw.io	9
Week 3	10
Frontend Interaction and Backend Foundations	10
JavaScript and PHP Basics	10
Fig 1: JavaScript Form Validation	10
Fig 2: Password Strength Checker	11
Fig 3: PHP Syntax Practice	11
Fig 4: Database Connection Script	12
Fig 5: Dynamic User Input Handling	13

WEEK 1

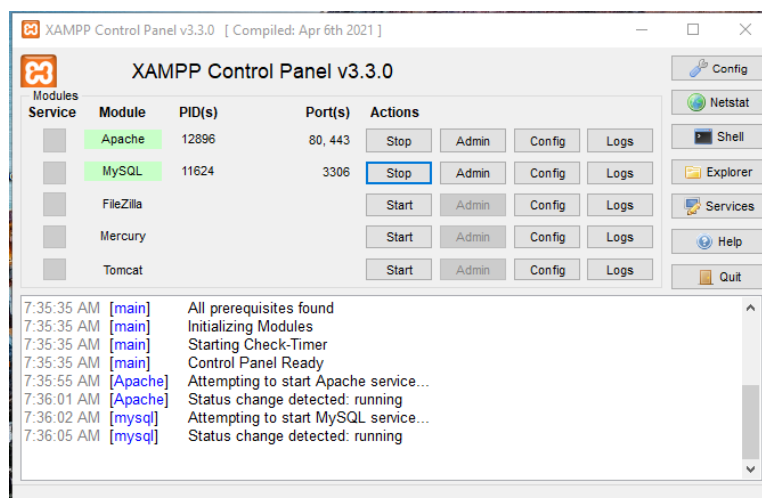
Local Environment Setup

Fig 1: Installation of XAMP



This Shows the XAMPP installer completing successfully on the local machine, confirming all core components Apache, MySQL, PHP, and phpMyAdmin were packaged and installed without errors

Fig 2: Apache and MySQL Running



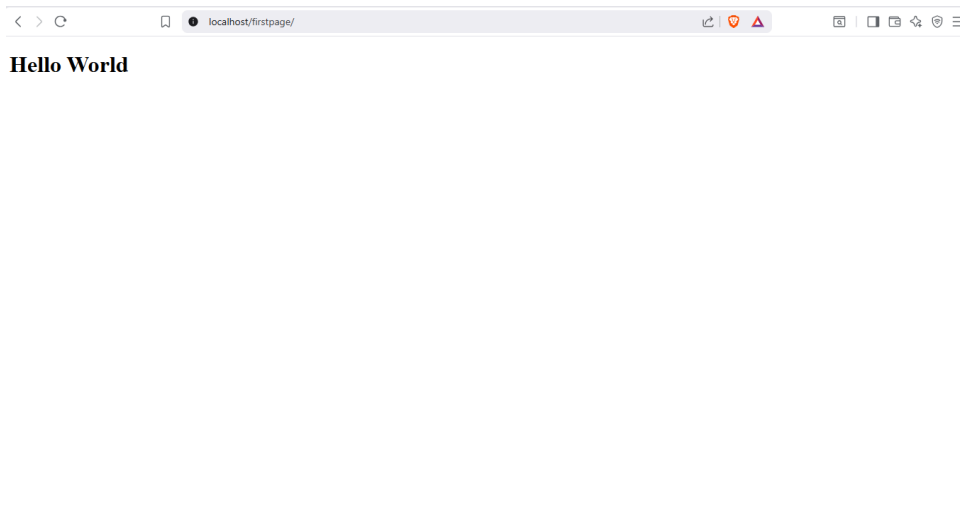
XAMPP Control Panel with both Apache and MySQL modules displaying a green "Running" status, confirming the web and database services are active and ready to handle requests.

Fig 3: Localhost Test Page



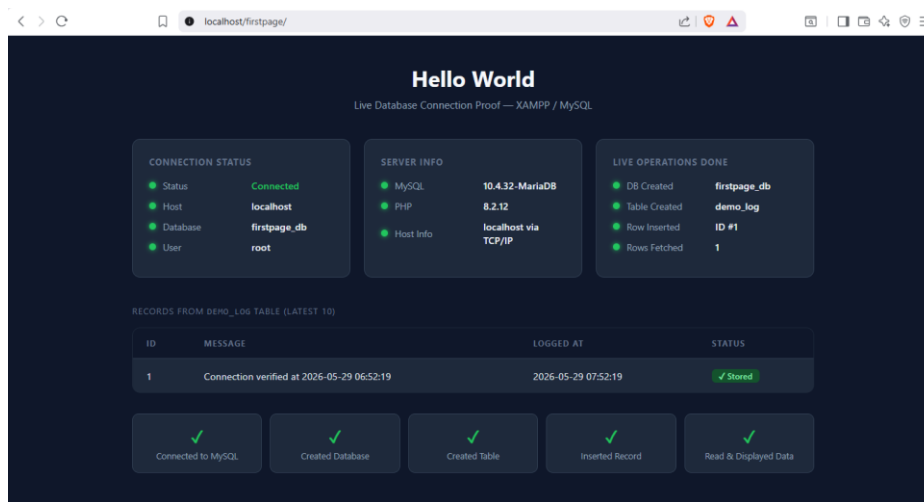
Shows the browser loading <http://localhost> and rendering the XAMPP welcome dashboard, confirming Apache is correctly serving pages from the htdocs root directory.

Fig 4: Hello World Test



This Shows a basic PHP or HTML file rendered in the browser displaying "Hello World", confirming the server correctly processes and outputs PHP scripts placed in the working project folder.

Fig 5: Database Connection Test



This is a successful connection between PHP and MySQL using `mysqli_connect()`, confirming Apache and MySQL services are properly configured and communicating within the XAMPP environment.

Lesson Learnt

During this week, I installed and configured XAMPP as my local development environment. The installation process set up Apache, MySQL, PHP, and phpMyAdmin successfully on my machine. I verified that both Apache and MySQL services were running correctly via the XAMPP Control Panel. Testing `http://localhost` confirmed Apache was serving pages from the `htdocs` directory. I then created a basic PHP file to display "Hello World", confirming the server processes PHP scripts correctly. Finally, I established a database connection using `mysqli_connect()`, which confirmed full communication between PHP and MySQL. The main challenge was ensuring both services started without port conflicts.

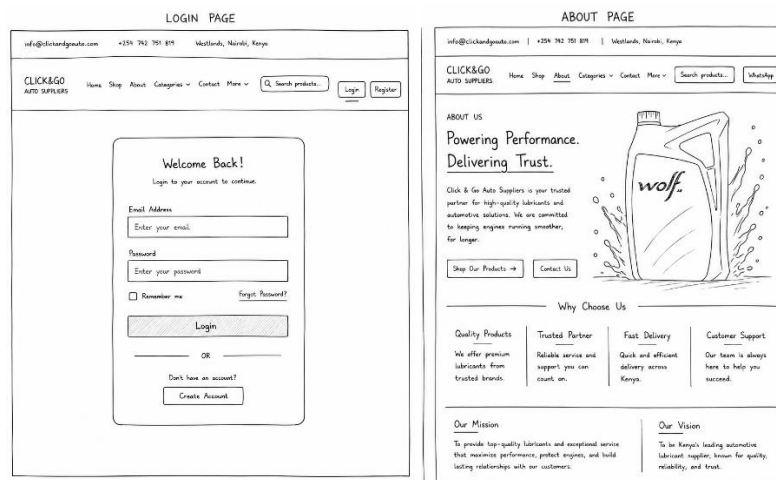
Week 2

User Interface Planning and System Design

Used Framer for this project

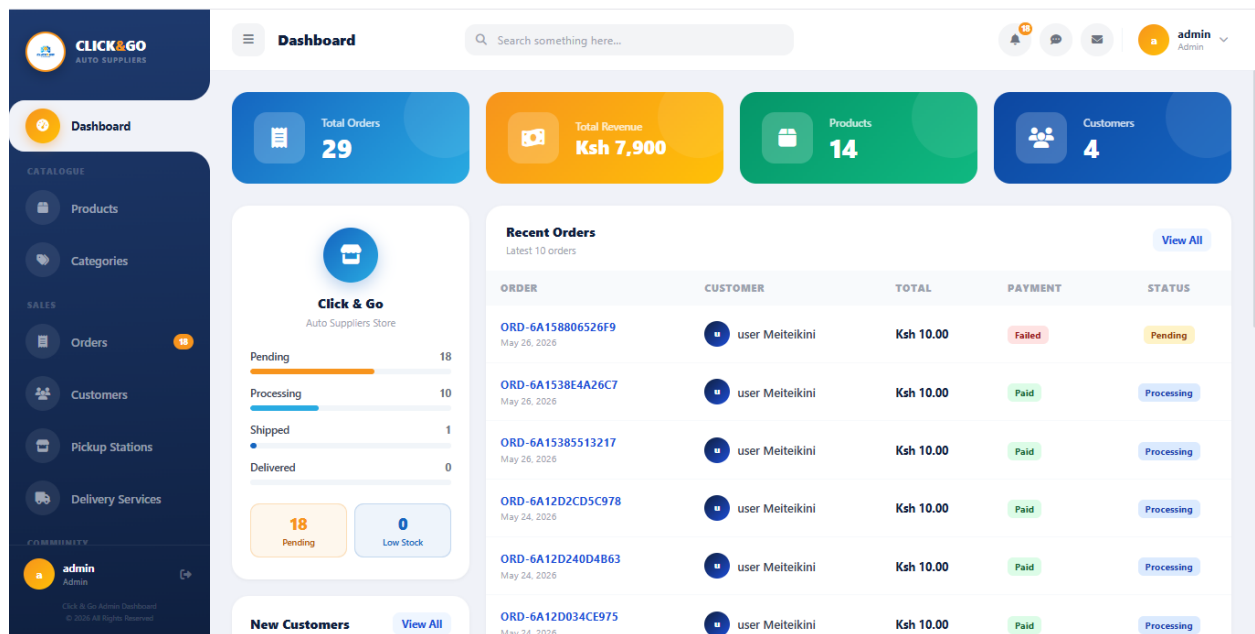
Wireframes and GUI Design

Fig 1: Hand-drawn Wireframe



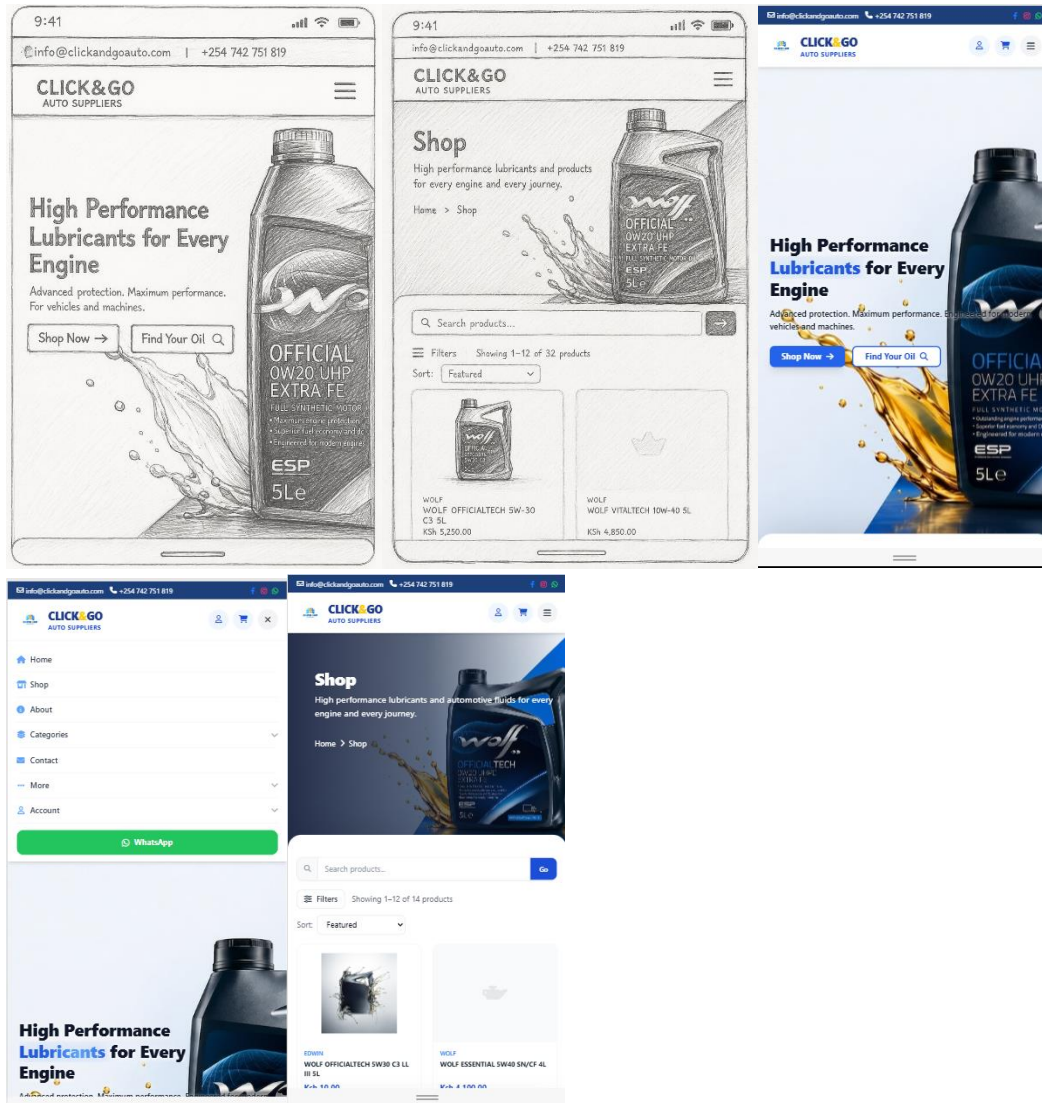
This are grafted hand-drawn sketch of the Login or aboutpage layout, illustrating the initial placement of key UI elements navigation bar, input fields, and call-to-action button before any digital design or coding began.

Fig 2: Dashboard Layout Design



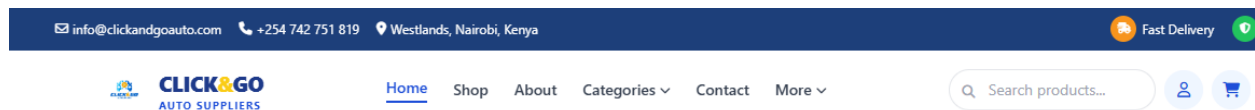
This is the planned dashboard structure, mapping out the positions of data panels, sidebar navigation, and content zones, ensuring a clear information hierarchy and logical flow before moving to digital prototyping.

Fig 3: Mobile View Mockup



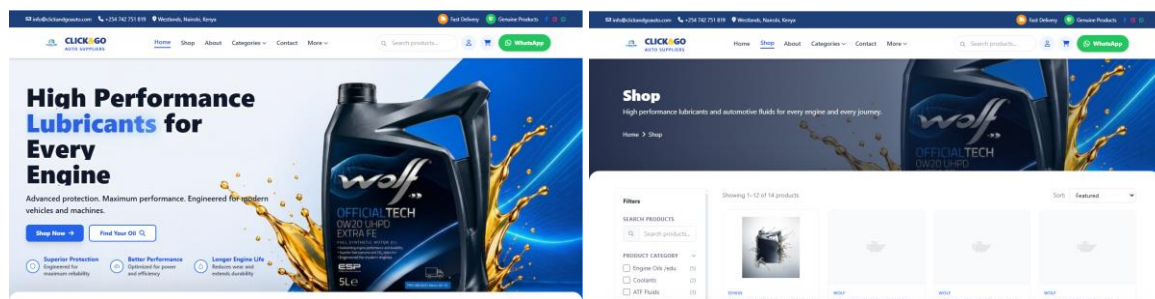
This is the mobile-responsive layout adaptation, demonstrating how the navigation collapses and content stacks vertically on smaller screens to maintain full usability across different device sizes.

Fig 4: Color Theme and Navigation Structure



This is the selected color palette and navigation flow diagram, confirming consistent brand colors were chosen early and that user journeys between pages were mapped to minimize unnecessary clicks and improve accessibility.

Fig 5: GUI Prototype in Figma/Canva/Draw.io



The figure above shows a high-fidelity screen prototype built in Framer presenting the finalized visual design of at least one key screen, ready to guide the actual PHP coding and layout implementation phase

What I learnt

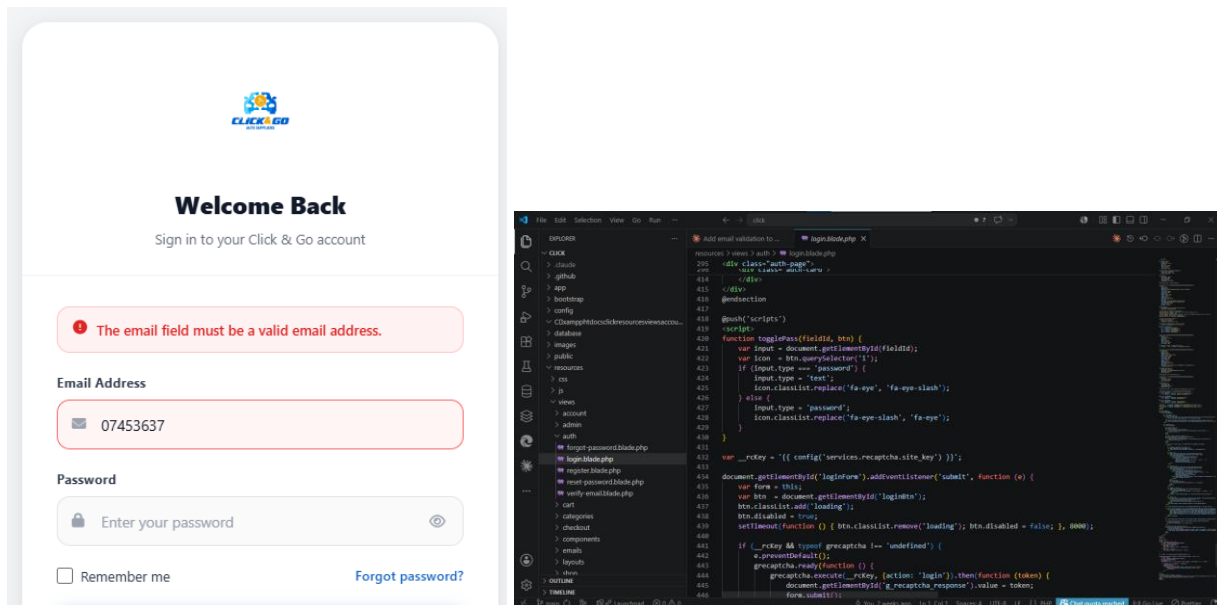
This week focused on planning the user interface before writing any code. I created hand-drawn wireframes for the login page and dashboard to map out element placement and navigation flow. A mobile mockup was designed to ensure responsiveness across devices. I selected a consistent color theme to improve visual identity and user experience. A high-fidelity prototype was then built in Figma to finalize the layout. The key challenge was balancing simplicity with functionality, particularly when designing the dashboard to minimize user clicks while keeping the interface scalable for future system modules.

Week 3

Frontend Interaction and Backend Foundations

JavaScript and PHP Basics

Fig 1: JavaScript Form Validation



This is a JavaScript validation script preventing form submission when required fields are empty or contain invalid input, confirming client-side data checks are working before any data is sent to the PHP backend.

Fig 2: Password Strength Checker

The screenshot shows a web browser at the URL `localhost/click/public/register`. The page is for 'CLICK&GO AUTO SUPPLIERS' and features a 'Create Your Account' form. The form includes fields for 'Full Name', 'Phone (optional)', 'Email Address', 'Password', and 'Confirm Password'. The 'Password' field is highlighted with a red border and a 'Very weak' strength indicator. Below the password field, there are four criteria: '8+ characters' (green dot), 'Lowercase letter' (green dot), 'Uppercase letter' (grey dot), and 'Number' (grey dot). At the bottom of the form, there is a checkbox for 'I have read and agree to the Terms & Conditions and Privacy Policy of Click & Go Auto Suppliers.'.

Shows a real time password strength indicator built with JavaScript, providing instant visual feedback as the user types, enforcing stronger password choices entirely on the client side before the form is submitted.

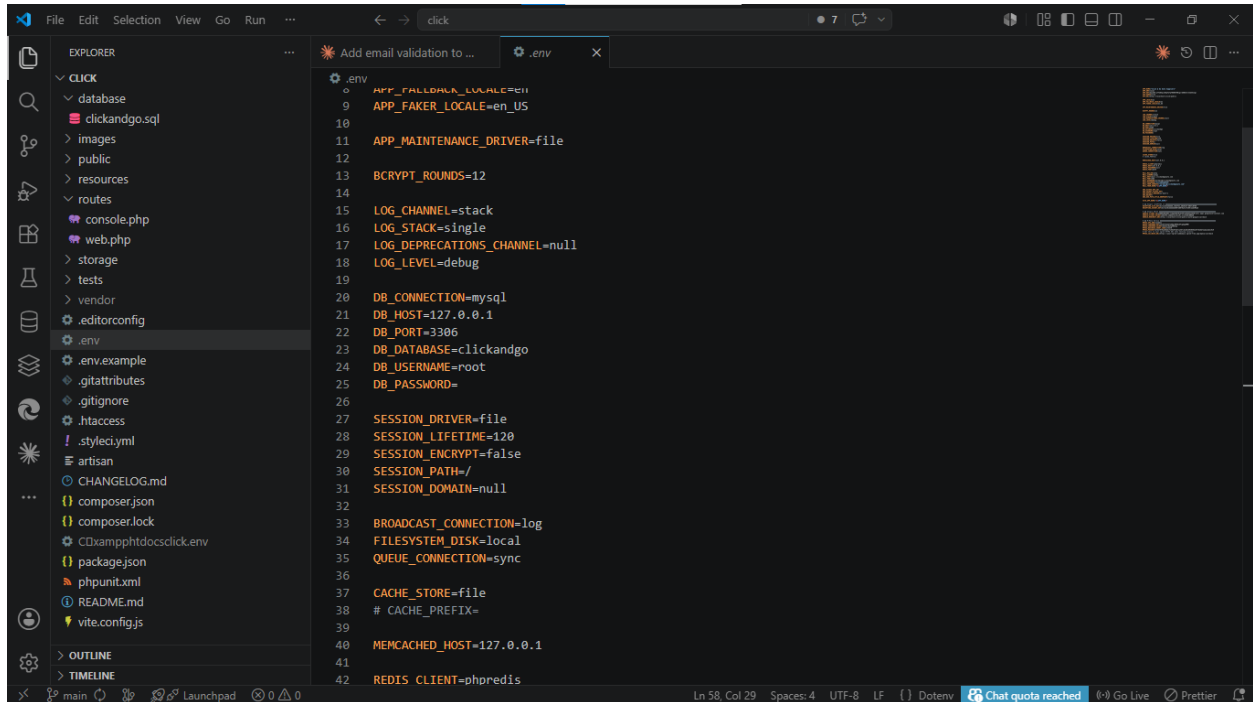
Fig 3: PHP Syntax Practice

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with files like `.claude`, `.github`, `app`, `bootstrap`, `config`, `CDxamppphpdocslickresourcesviewsaccountpartials`, `database`, `images`, `public`, `resources`, `routes`, `storage`, `tests`, `vendor`, `.editorconfig`, `.env`, `.env.example`, `.gitattributes`, `.gitignore`, `.htaccess`, `.styleci.yml`, `artisan`, `CHANGELOG.md`, `composer.json`, `composer.lock`, `CDxamppphpdocslick.env`, `package.json`, `phpunit.xml`, and `OUTLINE`. The code editor shows the `index.php` file with the following PHP code:

```
1 <?php
2
3 use Illuminate\Foundation\Application;
4 use Illuminate\Http\Request;
5
6 define('LARAVEL_START', microtime(true));
7
8 // Determine if the application is in maintenance mode...
9 if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
10     require $maintenance;
11 }
12
13 // Register the Composer autoloader...
14 require __DIR__.'/../vendor/autoload.php';
15
16 // Bootstrap Laravel and handle the request...
17 /** @var Application $app */
18 $app = require_once __DIR__.'/../bootstrap/app.php';
19
20 $app->handleRequest(Request::capture());
21
```

This is a PHP script open in VS Code displaying correct syntax structure including variables, conditionals, and echo output statements, confirming foundational server-side scripting skills before connecting to the MySQL database.

Fig 4: Database Connection Script



The image shows a screenshot of the Visual Studio Code editor. On the left, the Explorer sidebar displays a file tree for a project named 'CLICK'. The file tree includes folders like 'database', 'images', 'public', 'resources', 'routes', and 'tests', as well as files like 'console.php', 'web.php', 'storage', 'tests', 'vendor', '.editorconfig', '.env', '.env.example', '.gitattributes', '.gitignore', '.htaccess', '.styleci.yml', 'artisan', 'CHANGELOG.md', 'composer.json', 'composer.lock', 'CDxampphtdocs\click\env', 'package.json', 'phpunit.xml', 'README.md', and 'vite.config.js'. The '.env' file is selected and open in the main editor. The .env file contains the following configuration:

```
.env
0 APP_FALLBACK_LOCALE=en
9 APP_FAKER_LOCALE=en_US
10
11 APP_MAINTENANCE_DRIVER=file
12
13 BCRYPT_ROUNDS=12
14
15 LOG_CHANNEL=stack
16 LOG_STACK=single
17 LOG_DEPRECATIONS_CHANNEL=null
18 LOG_LEVEL=debug
19
20 DB_CONNECTION=mysql
21 DB_HOST=127.0.0.1
22 DB_PORT=3306
23 DB_DATABASE=clickandgo
24 DB_USERNAME=root
25 DB_PASSWORD=
26
27 SESSION_DRIVER=file
28 SESSION_LIFETIME=120
29 SESSION_ENCRYPT=false
30 SESSION_PATH=/
31 SESSION_DOMAIN=null
32
33 BROADCAST_CONNECTION=log
34 FILESYSTEM_DISK=local
35 QUEUE_CONNECTION=sync
36
37 CACHE_STORE=file
38 # CACHE_PREFIX=
39
40 MEMCACHED_HOST=127.0.0.1
41
42 REDIS_CLIENT=phpredis
```

The status bar at the bottom indicates the current position is 'Ln 58, Col 29' with 'Spaces: 4', 'UTF-8', 'LF', and 'Dotenv' encoding. It also shows a 'Chat quota reached' message and icons for 'Go Live' and 'Prettier'.

In this figure above shows the PHP script establishing a live connection to the MySQL database using `mysqli_connect()`, confirming that database credentials are correctly configured and the backend is ready to handle data read and write operations.

Fig 5: Dynamic User Input Handling

The figure consists of three screenshots illustrating dynamic user input handling in a web application.

Top Screenshot: Registration Form
The browser address bar shows `localhost/click/public/register`. The page title is "CLICK&GO AUTO SUPPLIERS". The registration form includes fields for Full Name (TEST), Phone (optional, +254746075436), Email Address (user@gmail.com), and Password (111Test@). The password strength is indicated as "Strong" with a checkmark. Below the password field, there are indicators for password requirements: 8+ characters, Uppercase letter, Lowercase letter, and Number. A "Confirm Password" field is also present. A checkbox for "I have read and agree to the Terms & Conditions and Privacy Policy of Click & Go Auto Suppliers." is checked. A "Create Account" button is at the bottom.

Middle Screenshot: Welcome Back Screen
The browser address bar shows `localhost/phpmyadmin/index.php?route=/sql&pos=0&db=clickandgo&table=users`. The page title is "CLICK&GO AUTO SUPPLIERS". The "Welcome Back" screen displays a message: "Account created! Please check your inbox to verify your email before signing in." Below this, there is a "Sign In" button and a "Forgot password?" link.

Bottom Screenshot: phpMyAdmin Database View
The browser address bar shows `localhost/phpmyadmin/index.php?route=/sql&pos=0&db=clickandgo&table=users`. The phpMyAdmin interface shows the "users" table with the following data:

id	name	email	email_verified_at	password	phone	role	address
4	user	user@gmail.com	2026-05-26 14:44:41	\$2y\$12\$8vD8S1Jw3G/zvqn3/g009i0QMph7FuUjgbu/p...	+254797242328	customer	NULL
10	iddris Rashid	iddrisrashid40@gmail.com	NULL	\$2y\$12\$8vD8S1Jw3G/zvqn3/g009i0QMph7FuUjgbu/p...	0724464248	customer	NULL
11	admin	admin@gmail.com	NULL	\$2y\$12\$8vD8S1Jw3G/zvqn3/g009i0QMph7FuUjgbu/p...	0797242328	admin	NULL
14	virginia	wamathavirgia56@gmail.com	NULL	\$2y\$12\$8vD8S1Jw3G/zvqn3/g009i0QMph7FuUjgbu/p...	0726157565	customer	NULL
15	Edwin Luke	edwinluke1999@gmail.com	NULL	NULL	NULL	customer	NULL
16	TEST	test@gmail.com	NULL	\$2y\$12\$8vD8S1Jw3G/zvqn3/g009i0QMph7FuUjgbu/p...	+254746075436	customer	NULL

PHP successfully receiving user input from an HTML form via `$_POST`, processing it server-side, and storing or displaying the result through MySQL, demonstrating a complete end-to-end frontend-to-backend data flow.

Weekly flow

By the end of this week, I introduced both frontend interaction and backend scripting. I implemented JavaScript form validation to prevent empty or invalid submissions, and built a password strength checker to improve security on the client side. On the backend, I practiced PHP syntax including variables, conditionals, and output statements. I then wrote a database connection script using `mysqli_connect ()` to link PHP with MySQL. Finally, I handled dynamic user input using `$_POST`, storing form data in the database. The main challenge was understanding how frontend validation and backend processing work together to create a secure and reliable data flow.